

TA DEMONSTRATION PRACTICE GUIDE

CSI 3140 Lab 5 — Secure Course Hub

Use this safely: Use only the local Moodle site and synthetic accounts. Do not show, copy, type into a slide, or record passwords, cookies, sesskeys, database credentials, or browser profiles.

1. Demo route at a glance

Stage	What the TA should see	Purpose
1. Start	Local Moodle opens at 127.0.0.1:8080; demonstration course is visible.	Shows a working local environment.
2. Structure	Plugin files under local/securecoursehub.	Shows this is a Moodle plugin, not a core modification.
3. Student	Student creates and views only an own request.	Shows authentication, course context, and ownership.
4. Denied action	Student B cannot perform a teacher operation or access Student A data.	Shows server-side authorization.
5. Teacher	Teacher updates status and response without page reload.	Shows authorized course management and JSON/fetch.
6. Security	Code and safe automated checks for CSRF, validation, XSS, and safe errors.	Shows security is enforced on the server.

2. Before the TA arrives

1. **Open Terminal in the lab folder.** — Say: “This is the project folder containing Docker Compose and the plugin source.”

```
cd "/Users/justinevaristo/Documents/CSI3140 Lab 5"
docker compose up -d
docker compose ps
```

2. **Open the site.** — Say: “Moodle is running locally, not publicly, at this address.”

```
http://127.0.0.1:8080
```

3. **Confirm the course and user sessions before the demo.** — Say: “I will use the normal course navigation to open Secure Course Hub. If a browser is already signed in, I log out through the user menu before switching roles.”

Practical tip: Use a normal browser window for one role and a private/incognito window for another role. That keeps the Moodle sessions separate without exposing session values.

3. Plugin folder walkthrough

Open the project folder in VS Code or Finder, then expand local/securecoursehub. Explain the following files in this order:

File or folder	What to say
version.php	Declares the Moodle component name and version so Moodle can install and upgrade the plugin.
db/install.xml	Defines the custom request table: course, owner, title, description, status, response, and timestamps.
db/access.php	Defines the course-context capabilities for viewing, creating, and managing requests.
index.php	Protected page controller and rendered interface. It requires login, establishes course context, checks capabilities, and handles regular student forms.
classes/local/request_service.php	Central business and database layer. It validates inputs, checks ownership/course association, and uses Moodle's \$DB API.
ajax.php	Protected JSON endpoint for teacher status/response updates. It repeats login, sesskey, context, capability, and record checks.
amd/src/dashboard.js	Browser-side fetch logic. It sends only the necessary same-origin JSON request and updates visible text without a reload.
lib.php and lang/en/	Adds the course-navigation entry and stores user-visible language strings.

Suggested one-sentence summary: “The page and AJAX endpoint are thin entry points; the service class holds the resource-level rules and uses Moodle’s Database API.”

4. Student demonstration: create and view an own request

- Log in as Student A.** — Say: “I am using a synthetic Moodle account; the password remains only in my local .env file.”
- Open the demonstration course and choose Secure Course Hub from the normal course navigation.** — Say: “The plugin is reachable from Moodle’s usual course interface, not only from a manually typed URL.”

6. **Create a synthetic request.** — Say: “I will use a non-personal title and description, then submit it. The server sets the owner from the authenticated Moodle user rather than accepting an owner id from the browser.”
7. **Show My requests.** — Say: “The student sees only records where both the course and user id match the current authenticated user.”
8. **Optionally edit or delete the new open request.** — Say: “Students can change or delete only their own requests while they are still open; the service checks that rule again on the server.”

5. Unauthorized student operation

Use this as the negative authorization demonstration. Do not try to bypass the application manually in a way that exposes tokens or cookies.

9. **Log in as Student B and open Secure Course Hub for the same course.** — Say: “Student B does not see Student A’s request because the own-request query is scoped to the authenticated user id.”
10. **Run the prepared safe negative-test script in Terminal.** — Say: “The script uses the local test configuration but prints only pass/fail results, never session values.”

```
cd "/Users/justinevaristo/Documents/CSI3140 Lab 5"
bash scripts/test_ajax_access_controls.sh
```

Expected output includes: student_teacher_operation=pass, teacher_unauthorized_course=pass, and teacher_missing_record=pass.

What to say: “Hiding a teacher button would not be security. The AJAX endpoint independently requires the manage-course-requests capability in the record’s course context. A student request receives HTTP 403 and no record changes.”

6. Teacher demonstration: authorized course management

11. **Log in as teacher1 and open the same course’s Secure Course Hub page.** — Say: “The teacher has the management capability in this course context, so the course request queue is visible.”
12. **Choose a synthetic request, change its status, add a short synthetic response, and choose Save course update.** — Say: “This is the required state-changing JSON/fetch operation.”
13. **Point out the success feedback and changed status/response in the same page.** — Say: “The visible page updates without a full reload.”

Keep the demo clean: Use synthetic request text. Do not alter a real person’s data or display private information.

7. Code walkthrough: capability, context, and ownership checks

Where	What it proves	Short explanation
db/access.php	Capability definitions	Students can view/create; editing teachers and managers can manage requests in a course.
index.php	Page protection	require_login(\$course) authenticates the session; course context is created; require_capability(viewown, context) gates the page.
index.php	UI path selection	has_capability(managecourserequests, context) decides whether the teacher queue is shown. This is usability only; it is not the final security decision.
ajax.php	Teacher endpoint protection	It requires login, validates the JSON and sesskey, creates course context, and requires managecourserequests before updating.
request_service.php	Resource-level rules	Students must own the record and it must be open. Teachers must manage the exact course attached to the record.

```
// index.php: page access
require_login($course);
$context = context_course::instance($courseid);
require_capability('local/securecoursehub:viewown', $context);

// ajax.php: management update
if (!confirm_sesskey($payload['sesskey'])) { /* reject safely */ }
require_capability('local/securecoursehub:managecourserequests', $context);
```

8. JSON/fetch interaction in browser developer tools

14. **While signed in as teacher1, open Developer Tools → Network and clear the list.** — Say: “I will inspect only the request name, status, and safe response shape; I will not display credentials, cookies, or the sesskey.”
15. **Perform the teacher update once more.** — Say: “The browser sends one same-origin POST request to ajax.php with JSON.”
16. **Select the ajax.php request and point out the successful HTTP result and structured JSON success response.** — Say: “The server validates the request before changing the record, then returns only the updated request values and a message.”
17. **Point to amd/src/dashboard.js.** — Say: “fetch() sends the request;.textContent updates the status and response safely rather than inserting untrusted HTML.”

```
const request = await fetch(endpoint, {
  method: 'POST',
  headers: {'Content-Type': 'application/json'},
  body: JSON.stringify({action, courseid, id, status, response, sesskey: M.cfg.sesskey})
});
// On success, status and response are changed with textContent — no full reload.
```

Do not reveal secrets: Do not expand or screenshot request headers/cookies, hidden form fields, or JSON values that contain a sesskey. The demonstration is about the control, not the secret value.

9. CSRF (sesskey) demonstration

Say: “CSRF protection proves that a state-changing request originated from a page Moodle issued for this logged-in session. Moodle supplies a sesskey; the server validates it again before any write.”

```
cd "/Users/justinevaristo/Documents/CSI3140 Lab 5"
bash scripts/test_invalid_sesskey.sh
```

Expected result: `invalid_sesskey_test=pass`. The test sends an intentionally invalid placeholder, expects HTTP 400, and verifies the request-table count did not change. It does not print a real sesskey.

Code locations to show: `index.php` uses `require_sesskey()` for regular forms; `ajax.php` calls `confirm_sesskey()` for JSON before the capability and update logic.

10. Input validation and XSS-safe output

18. **As Student A, create a temporary request using a literal XSS-style title.** — Say: “I will enter the characters `<script>alert(1)</script>` as plain text. This is synthetic test content only.”
19. **Show that the characters are displayed literally and no alert runs.** — Say: “The plugin treats user text as data, not executable markup.”
20. **Delete the temporary open request after the demonstration if desired.** — Say: “This keeps the local demonstration data tidy.”

What to say: “The server validates presence, length, allowed status values, course association, ownership rules, and the open-state rule. PHP output is escaped with Moodle’s `s()` function; JavaScript uses `textContent` instead of `innerHTML`.”

```
// request_service.php keeps literal markup as data; output is escaped later.
$title = clean_param($title, PARAM_RAW_TRIMMED);

// dashboard.js renders returned text safely.
statusdisplay.textContent = result.request.status;
responsedisplay.textContent = result.request.response || feedback.dataset.emptyResponse;
```

11. Short answers for TA questions

Question	Answer to practise
----------	--------------------

What is authentication?	Authentication answers “who is this user?” Moodle performs it through its existing login and server-side session. The plugin uses <code>require_login()</code> rather than its own password database.
What is authorization?	Authorization answers “is this authenticated user allowed to perform this action here?” The plugin checks Moodle capabilities in the relevant course context.
What are role and capability checks?	A role, such as student or editing teacher, is Moodle’s assignment. A capability is the permission the plugin checks, such as <code>managecourserequests</code> . Roles can be configured, so code checks capabilities rather than trusting a role name from the browser.
What is ownership?	Ownership is a resource-level rule. Even a student who may create requests can update or delete only a record whose <code>userid</code> matches the authenticated user, and only while it is open.
Why check the course context?	A teacher may manage one course but not another. The endpoint checks the capability in the request record’s course context, not just whether the user is a teacher somewhere.
Why use Moodle’s Database API?	It avoids unsafe SQL concatenation, uses Moodle conventions, and keeps database access in the service class.
How is the JSON endpoint protected?	<code>ajax.php</code> requires login, validates JSON fields and <code>sesskey</code> , builds the course context, requires the management capability, confirms the record belongs to the course, then calls the service-layer update.
What privacy decision did you make?	The plugin stores only the data needed for a course request: course/user identifiers, request text, status, response, and timestamps. It does not collect passwords, cookies, <code>sesskeys</code> , or extra profile data.
What is one known limitation?	This lab version has no email/notification workflow or automated retention deletion. It uses synthetic demonstration data, and the local test data should be removed/reset after the lab.

12. Final rehearsal checklist

- Docker Compose is running and the site opens at `http://127.0.0.1:8080`.
- The demonstration course opens and Secure Course Hub is available from normal course navigation.
- A synthetic open request exists for the teacher-update demonstration.
- You can explain each main file and find `index.php`, `ajax.php`, `db/access.php`, `request_service.php`, and `dashboard.js` quickly.
- You have rehearsed the student own-request flow and the teacher update flow.
- The safe negative-test scripts are ready: `bash scripts/test_ajax_access_controls.sh` and `bash scripts/test_invalid_sesskey.sh`.
- Developer Tools is set to Network; you will avoid showing headers, cookies, passwords, and sesskeys.
- You can explain authentication, authorization, capability checks, ownership, one privacy choice, and one limitation in your own words.

After the demo: Do not submit Docker volumes, Moodle core, `moodledata`, `config.php`, `.env`, passwords, session material, or browser profiles. The deliverable is the plugin folder plus the required safe evidence and report.

13. Thirty-second closing summary

Practise this closing in your own words: “Secure Course Hub is a Moodle local plugin for course-related help requests. Moodle authenticates users and manages their sessions. The plugin authorizes each action through course-context capabilities and resource-level ownership rules, then uses Moodle’s Database API for safe persistence. Student forms and the teacher JSON endpoint both validate CSRF protection and input on the server. The teacher update demonstrates `fetch()` and a dynamic page update, while the negative tests show that hidden UI controls are not the security boundary.”

14. Quick recovery if something is not ready

Site unavailable: Run `docker compose ps`. If Moodle is not running, run `docker compose up -d` and wait a moment before refreshing `127.0.0.1:8080`.

Wrong signed-in user: Use the top-right Moodle user menu to log out. A private/incognito window is useful for switching roles.

JavaScript changes missing: Deploy the plugin update to the container and clear Moodle’s cache before retrying.